THM - Notes

Windows Fundamentals

`msconfig` - Systems Config Util - handles General (startup), Boot, Services, Startup (now handled by `taskmgr`) & Tools (to launch some specific service).

`control.exe` opens the control panel.

`UserAccountControlSettings.exe` is the UAC manager.

Some apps that are attached to ControlPanel may have a different extension. Example : `wscui.cpl` -> which opens up the Security & Maintenance section of ControlPanel.

Other things, can be attributed as `.msc` - which are the services running via Management console. One such service - `compmgmt.msc` -- which can handle a lot of tasks and contains a lot of services/link to them. `Win+R>compmgmt.msc` or `Win+X>Comp Management`
`compmgmt.msc` can make tasks + bind them. It also contains `lusrmgr.msc` and even `perfmon`
WMI and WMI-CLI (Windows Mgmt Instrumentation) was deprecated and superseded by Powershell.
`msinfo32.exe` is capable of gathering all HW info along with SW Components.
There's also `resmon.exe` - Resource Manager which taskmgr but on steroids.
`regedit.exe` - reg editor

CMD Commands : `hostname`, `whomai`, `ipconfig`, `/?` (used as 'man' command in linux, although some commands would want `'help'` instead), `cls`, `netstat`, `net` (manage n/w resources) [net is used for user-management as well]
VSS or Volume Shadow Copy Service handles the restore point and all.

Active Directory Basics

Windows Domain is a group of users & computers under the administration of some business. The centralised repo of all those machines is called **Active Directory** & the server that runs this AD is called **Domain Controller** (DC).
This way, you can centralise ID mgmt and have a unified method of managing security policies.
Examples - the university/school network that gave you one user ID/pw and you can login on any machine and still have your stuff synced + restrictions (like control panel) still in place.
DS is domain service i.e the catalogue storing info about who all are in the AD (users,

groups, PC, printers, etc.)
Users : also called *'Security Principals'*, they basically can act upon the resources on the network. People (humans) and Services (IIS or MySQL) both fall under this.
Machines : also considered Security Principals, but they are an object created for any and every computer that joins the network. They do have an account this way but very limited. No one else is supposed to log in to one machine's account but ofc they can. Naming scheme is simple. It uses the computer's name followed by a $ -> `AdminPC` would have a machine name as `AdminPC$`.
Security Groups : Basically like User Groups - having rights to file/folders with respective permissions. Also 'Security Principals'.
Examples of some domains present in a group ->
Domain Admins - admin priv over the entire domain. Can administer anything.
Server Ops - Can administer DC but cannot change any admin group memberships.
Backup Ops - To perform data backups on computers. Can ignore any file permissions.
Account Ops - Create/modify other users in the domain.
Domain Users - All existing user accounts in the domain.
Domain Computers - All existing computers in the domain.
Domain Controllers - All DCs on the domain

Windows already has the "Active Directory Users & Computers" tool to handle all this. In this we have Organizational Units (OU) which are just container objects that allow us to classify users & machines. 1 User can only be in 1 OU, and each OU can have totally difft+separate permissions (Sales, IT, Marketing, etc).
Ofc there are default containers too (BuiltIn, Computers, DC, Users, Managed Service Accounts).
Basically, quoting THM directly :

- **OUs** are handy for **applying policies** to users and computers, which include specific configurations that pertain to sets of users depending on their particular role in the enterprise. Remember, a user can only be a member of a single OU at a time, as it wouldn't make sense to try to apply two different sets of policies to a single user.
- **Security Groups**, on the other hand, are used to **grant permissions over resources**. For example, you will use groups if you want to allow some users to access a shared folder or network printer. A user can be a part of many groups, which is needed to grant access to multiple resources.

Managing users in AD
Deleting OUs or Users :
By default, OUs are protected against accidental deletion. To fix this, enable Advanced Features in the view menu, then Rt-click the OU you want to delete > properties > object & un-check the "Protect object from accidental deletion"

Delegation :
Allowing some users have control over some OUs is delegation. Example : IT being given the priv to reset other low-level user's passwords. For this, Rt-click on the OU you want to hand over the control of, then fill in the user name of that person (IT guy in our case) followed by the tasks selection for delegation.
In such cases, we can RDP to this IT member who can run a powershell script to remotely change the password of any of the users and optionally prompt a password reset on logon.
`PS > Set-ADAccountPassword sophie -Reset -NewPassword (Read-Host -AsSecureString 'New Password')`

Managing computers in AD
By default, all machines that join a domain (except for the DCs) will be put in the container called "Computers". The entries might have their own naming convention like 'LPT-SOPHIE' or 'PC-MARK'.

Group Policies
The main idea of organizing users and computers into separate OUs is to have customized Group Policies for each of them. Situated inside the "Group Policy Management" tool, we have the GPO or Group Policy Objects -- they simply are the collection of settings that can be applied to OUs. They can be used to apply settings/rules to computers.
(Policies are created under the GPO section but linked in the general section).
Examples like Default Domain Policy or RDP Policy, etc.
GPOs are distributed to the network via a network share called `SYSVOL`, which is stored in the DC. All users in a domain should typically have access to this share over the network to sync their GPOs periodically. The SYSVOL share points by default to the `C:\Windows\SYSVOL\sysvol\` directory on each of the DCs in our network.

Authentication Methods
On Windows domains, credentials are stored with the DC -- as these are now Domain Credentials -- so the service talks to the DC for auth via any of the 2 protocols :

Kerberos -> default protocol in any recent domain + used by recent versions of Windows.
Standard behavior is to assign tickets at log in -- tickets act as a proof of prev auth. Basically it involves the username + timestamp in the requst, encrypted with a key. The Key Distribution Center (KDC) will send back to the usrer, a TGT - ticket granting ticket - and a session key. Then the client, while connecting to a service, asks for a TGS - ticket granting service, which are service specific - which is done when the user sends their username+timestamp encrypted using the Session key + send the TGT and SPN (service principal name). In return, the KDC sends us a TGS along with a service

session key in the response. Now the user re-encrypts the TGS with service-owner hash and finally connects to the service

NetNTLM - legacy, mainly for compatibility. Client sends an auth req -> server generates a random number & sends it as a challenge to the client -> Client then combines NTLM password hash with the challenge to generate a resp to the challenge & sends it back to the server for verification. The server forwards the challenge & the response to the DC for verification. The DC uses the challenge to recalculate the response and compares it to the original response sent by the client. If they both match, client is auth-cated & server forwards the auth result to the client.

Trees, Forests & Trusts

Tree can be formed with a sub-domain like AD network. Example : `thm.local` is our main AD network, and `uk.thm.local` and `eu.thm.local` will be the sub-domains.
Forest is when 2 or more trees are union-ized as one big network. So all trees will be filled with their respective sub-domains in it.
For this, a new security group is introduced - **Enterprise Admins**.
Trust Relationships are required when different trees need to access files stored on each other. Example : UK THM needs to access something in MHT Asia or EU. So we need a one-way trust relationship. Trust is one direction, access is the other. Two way is also possible. Does not mean everyone is now family, you still need per-user auth.

Bash Basics

(Most is covered elsewhere)
In bash, you can debug by writing `set +` and `set -` inside your code making a block, then on the terminal run `$ > bash +x ./script.sh`. This will show a '+' for the lines that worked perfectly and '-' for the ones that failed.
`$1`, `$2` etc are all parameters that can be sent when calling the script. Inside the script they can be used like normal variables. Input is still handled by `read`. However, `$0` is used to fetch the script name itself i.e 0 parameter.
`$#` is used to get the number of args supplied while calling the script.

Arrays

`transport=('car' 'train' 'bike' 'bus')`
To use : `echo "${transport[@]}"` -- @ is for all elements, can be replaced by numeric digits.
To unset/delete an element -- `use unset transport[1]`.
To set any element to something else, the same syntax follows -- `transport[1]='truck'`

Conditionals

```
if [condition compare condition]
then
body_of_if
```

```
else
body_of_else
fi
```

Conditionals like `-eq , -ne , -gt , -lt , -ge`

Example code :

```
#! /bin/bash
filename=$1
if [ -f "$filename" ] && [ -w "$filename" ]
then
echo "hello" > "$filename"
else
touch "$filename"
echo "hello" > "$filename"
fi
```

//Here, `-f` checks if the file exists & `-w` checks if its writable. `-r` for readable and `-d` to see if it's a directory.

(Part of Network Fundamentals Module)

LAN Basics

Subnetting :
We know, IP Addrs are split into 4 octets, ranging from 0-255. Subnet is just a sub-category, also ranging from 0-255, 4 bytes(32 bits). Subnets use IP Addr in 3 different ways -> ID the network addr, ID the host addr & ID the default gateway.
Examples ->
N/W Addr - Identifies the actual start of the N/W and its existence. Device with 192.168.1.100 will be on the N/w identified by 192.168.1.0
Host Addr - Used to identify a device on the subnet. A device will have the N/W Addr of 192.168.1.1
Default Gateway - A special addr assigned to a device on the N/W that is capable of sending info to another network. Any data that needs to go to a device that isn't on the same network (168.1.0) will be sent to this device. These devices can use any host addr but usually use either the first or the last addr in the network (.1 or .254)

ARP is the link b/w the MAC Addr & IP Addr of any device on the network. ARP Req (packet) is sent -- a message is broadcasted on the N/w asking the MAC of the IP in question. Only the valid party is allowed to respond via the ARP Reply (packet).

DHCP is responsible for assigning the IP Addresses. When not manually assigned, a new device sends out a `DHCP Discover` req to get any DHCP servers on the N/w. DHCP Server then replies back with an IP Addr via the `DHCP Offer` reply. Then device gets it, sends back a confirmation via `DHCP Request` to the server. Lastly, the DHCP

server sends the reply acknowledging via `DHCP ACK`. Device only sends 2 messages - Discover & Request. Server has the other 2, OFFER & ACK.

OSI Model

Open Systems Interconnection.
7 Layers. From 7 to 1 -> **Application, Presentation, Session, Transport, Network, Data Link & Physical**. In OSI, devices can have different functions & designs on a network while communicating with other devices. This model provides a framework dictating how all networked devices will send, receive & interpret data. Encapsulation - pieces of info get added to data.
Observing all layers ->

1. Physical : Ethernet cables for example.
2. Data Link : focuses on physical addressing of the transmission. It receives a packet from the network layer (including the IP) and adds the MAC addr of the receiving endpoint. NIC handles this MAC thing. Remember, MAC cannot be changed but can be spoofed. This layer presents the data in a suitable format for transmission.
3. Network : Routing & re-assembly of data. Optimal path via OSPF (Optimal shortest path first) & RIP (routing info protocol) -- shortest (least devices to hop), reliable (minimal packet loss) & faster physical connection (fibre vs copper). Everything is dealt via IP Addrs & devices here (router etc) are called Layer-3 devices.
4. Transport : Actual transmission. Methods - always 2 there are - TCP & UDP.
   A. TCP -> Reserves a constant connection b/w the 2 devices for the amount of time it takes for data to be sent & received + incorporates error checking (helps session layer). It guarantees the accuracy of data but requires a reliable connection b/w the 2 devices (a small chunk missing can render the entire chunk useless). It can handle device-sync to prevent flooding, but any slow connection will bottleneck the other device as the connection will be reserved on the receiving end. Slower than UDP bcz devices work a lot harder here. USE CASES : file sharing, browsing, email.
   B. UDP -> Not as advanced as TCP. It doesn't care if the data is received. Leaves the Application Layer (user end) to decide if there's any control over how quickly the packets are sent. It is flexible to SDEs. Does not reserve a continuous connection on a device as TCP. Unstable connection here results in a terrible experience for the user. USE CASES : ARP, DHCP, small pieces of data being sent, large video streaming.
5. Session : When data is formatted/translated from Layer-6, Layer-5 i.e Session layer will begin to create & maintain the connection to the computer for which the data is destined. Connection established -> session created. This layer is also responsible

for closing the connection on a timeout or error. Can also create checkpoints -- saves bandwidth. Unique in nature -- data cannot travel over different sessions.

6. Presentation : The translator. Here, standardization takes place. Back-and-forth with Layer-7. Example - email client varies but we still get the same thing, thanks to Presentation layer. HTTPS occurs at this layer.
7. Application : User-facing. Here, protocols & rules are in place to determine how the user should interact with data sent/received. USE CASES : Everything like email clients, browsers, file browing (FileZilla), DNS, etc.
   Order -> Phy-DL-NW-TR-SS-PRS-APP (Physical-D-NTS-PA)

Packets & Frames

What are they?

Both are small pieces of data. Packet is under layer-3 i.e Network -- containing IP header & payload. Frame is under layer-2 i.e Data Link -- encapsulates the packet & adds additional info such as the MAC Addr.
Analogy - Mailing a letter through post. Envelope is the frame, here used to move the letter i.e packet. Once the recipient opens the envelope, the can see the letter itself. This process is called 'encapsulation'.
Packets are efficient at communication across networked devices. Small pieces hence less chance of bottle-necking. Eg : Loading an image, instead of the whole image, small pieces are sent and then reconstructed on your local machine. The structure of a packet may vary depending upon the type.
Some notable headers ->
`Time to live` : lifespan/expiry timer of the packet
`Checksum` : integrity checking for TCP/IP. Notifies of tampering.
`Source Addr` : IP of where it was sent from, so that data knows where to return to.
`Dest Addr` : IP of where it's being sent to, so that data knows where to travel next.

TCP/IP (3-way handshake)

TCP/IP consists of 4 layers of its own-> Application, Transport, Internet & Network Interface.
(A.T.I.Ni). TCP uses these 4 layers to ensure that data is received at destination. TCP is connection-based -- i.e a connection needs to be established first. Each layer embeds some info in the packet.
A TCP packet has various sections called headers :
`Source Port (0-65535)` , `Destination Port` , `Source IP` , `Destination IP` , `Sequence Number` (every piece is given a random number), `Ack Number` (Seq No. + 1), `Checksum` (integrity hash), `Data` (where are the bytes stored), `Flag` (behavior - how it should be handled).

TCP has this special 3-way handshake. In total we have 6 messages ->
`SYN` - initial packet sent by the client. Used to initiate the connection + sync the devices.
`SYN/ACK` - receiving device sends it to acknowledge sync attempt.
`ACK` - can be used by either party to ack that a series of messages/packets have been rec'd.
`DATA` - actual bytes of data sent via this message
`FIN` - cleanly close the connection after completion
`RST` - abruptly ends all communications -- last resort & indicates a problem.

Process : 2 Devices A & B -> A sends `SYN` to B, B sends `SYN/ACK` back to A and then A sends `ACK` to B. Any sent data is given a Random Number seq & is reconstructed using this number sequence. Both devices must agree with the same number sequence for the data to be sent in the correct order. The order is agreed upon in the above 3 steps of the handshake. Basically,
`SYN` - Client : Here is my initial seq number (INS) to SYNchronise with (0)
`SYN/ACK` - Server : here's my INS to SNYchronise with (5000) & I ACKnowledge your INS (0)
`ACK` - Client : I ACKnowledge your INS (5000), here is some data which is INS+1 (0+1)
//This way, Client INS is going 0,1,2,3... and FNS will be 1,2,3...

Closing a connection ->
TCP closes the connection when the rec'r says it has rec'd all the data. TCP inherently reserves system resources, so closing it ASAP is best. To initiate the closure, device will send a "FIN" packet. Here,
A sends a `FIN` packet to B, then B sends `FIN` & `ACK` packets, then A replies with `ACK` and then the connection is closed.

UDP/IP

UDP is state-less protocol (no ACK system) + no need for a constant connection. No 3-way handshake. UDP can tolerate data loss + does not reserve a continuous connection on a device as TCP -- hence no safeguards such as data integrity.
UDP Headers : `TTL` (time to live), `Source IP`, `Dest IP`, `Source Port`, `Dest Port`, `Data`.
It feels one sided. Request sent from B to A and then A keeps on sending Response.

Ports
Ports are the point of exchange. Very specific. Like harbour, ships & ports. Ships line up and connect to a port at the harbour. A cruiser liner cannot dock at a port made for a fishing vessel & vice versa. Compatibility is defined by the ports. It if ain't compatible it can't park here. Ranging from 0 to 65535 but few apps have a fixed port.

`FTP - 21`, `SSH - 22`, `HTTP - 80`, `HTTPS - 443`, `SMB - 445`, `RDP - 3389`, etc.
Eg : `$ > nc 8.8.8.8 443`

Extending your Network

Port Forwarding
Configured at the router level, it is the most crucial step. Without it, everything is limited to its own network i.e Intranet. Example - a webserver running something on `Port 80` inside a network having `192.168.1.10` will not be public by default. For that it needs an IP ofc but also the services being port forwarded via that IP. So now this server will have a Public IP of `82.62.51.70`.

Firewall
Responsible for what traffic is allowed to enter/exit. Like border security. Can be configured by admin to permit/deny any traffic. It performs packet inspection to ask these questions -> `Coming from where? ;` Going to where? ; `For which port? ; &` What protocol is being used by this traffic?.`
Operates at Layer 3 & 4 i.e Network & Transport layer.
2 Types - Hardware (Fortinet) & software (Snort).
2 Categories ->
a) **Stateful** : Uses the *entire info from a connection* rather than inspecting individual packets. This will determine the behavior of a device *based upon the entire connection.* More resource-intensive but has dynamic decision making.
Eg : FW could allow the first parts of a TCP handshake that would later fail. If a connection from a host is bad, it will block the entire device.
b) **Stateless** : Uses *static set of rules to determine whether individual packets are acceptable or not.* Less resource-intensive, but also dumber. Great for tackling huge data like DDOS.
Eg : Device sending a bad packet will not necessarily mean that the entire device is blocked. These ones are only effective as the rules that are defined withing them. If a rule is not exactly matched, it is effectively useless.

VPN Basics
VPN works on the fundamental principle of making a secured end-to-end comms line via tunneling b/w two devices.
For instance - 2 offices can connect securely and directly via a VPN. Like individually, you can still be a part of your network but also be a part of that VPN to communicate.
VPN allows Networks in different geo-locations to be connected (best for multiple office branches) ; It also offers privacy via encryption & Offers anonymity (blocks your ISP from viewing your traffic).
Tech :
PPP - used by PPTP, used for auth + encryption. On it's own cannot leave a network

(non-routable)
PPTP - Point-to-point Tunneling Protocol - allows data from PPP to travel & leave the network. Easy to set-up + supported by most devices BUT is weakly encrypted.
IPSec - encrypts data using existing IP framework. Difficult to set-up BUT boasts a strong encryption + supported on many devices.

LAN NW'ing Devices
a) Router : Connect networks & pass data between them. Done by 'routing'. Operate on Layer-3 i.e Network layer. When many routers are present, the data takes optimal path automatically albeit governed by existing routing protocols.
b) Switch : Dedicated device meant for connecting many devices (3 to 63) via ethernet cables. Operate on both Layer-2 & 3 altho exclusive i.e Layer-2 switch won't operate at Layer-3.
Example -> Layer 2 is very simple. Layer-3 acts as a router as well. How? It segments via VLAN (`VLAN1` via `192.168.1.1` & `VLAN2` via `192.168.2.1`). This still treats them separate and they can still get the actual internet via the router -- altho this N/w segmentation will determine the comms rule for all devices.

Sample Simulation :
2 Machines. PC1 connected to Switch1 which is connected to Router. PC3 connected to Switch2 also connected to Router.
Sending a packet from PC1 to PC3. First a handshake packet is sent, followed by a SYN packet from PC1 to PC3. PC1 tells ROUTING (Switch1) that PC3 is not on his network, so ROUTING sends an ARP REQ to the router to first identify itself as the Router and then tell where PC3 is. Router tells him that and PC3 receives the SYN packet from PC1, sends back a SYN/ACK packet to PC1. PC1 recevies it and sendds back an ACK packet to PC3. PC3 recevies it from PC1 & handshake is complete. Post which, the TCP packet safely & directly travels from PC1 to PC3. After receiving the packet, PC3 sends an ACK to PC1. For that, another handshake is started b/w PC1 & PC3 -- SYN sent from PC1 to PC3. PC3 receives it and sends SYN/ACK to PC1, who then receives/acknowledges it and sends ACK to PC3. PC3 receives it.

DNS in Detail

Domain Name System. Converting IP Addr numbers into read-able websites.

Domain Heirarchy
Root Domain > Top Level Domain > Second-Level Domain -- Subdomains
TLD : Also 2 sub-types. gTLD (generic = .com, .org, .edu, .online, .club, .biz) & ccTLD (Country code = .ca, .co.uk, .in, .us).
SLD : The domain preceding the TLD is called SLD. Eg : tryhackme.com has TLD as .com and SLD as tryhackme. SLD is limited to 63 chars + TLD can only use a-z 0-9 and hyphens.

// Sub-domains have the same character limit, but no limit on the number of sub-domains that can be made.

Record Types
Many types of DNS records exist.
`A Record` : For IPv4 : 104.26.10.229
`AAAA Record` : For IPv6
`CNAME Record` : These resolve to another domain name. Example - 'store.tryhackme.com' returns a CNAME record 'shops.shopify.com' which further requires another DNS req for the IP Addr.
`MX Record` : Resolve to addr of the servers that handle the email for the domain you're querying. These records come with a priority flag -- telling the client in which order to try the servers (perfect for switching to backups)
`TXT Record` : Free text fields where any text-based data can be stored. Can have multiple uses but most common is to list servers that have the auth-y to send an email on behalf of the domain, or even verify the ownership of the domain.

Making a DNS Request
5 Steps :

1. When you req a domain name, your PC first checks its local cache to see if you've previously looked up the addr recently. False ? A req to your recursive DNS server will be made.
2. A Recursive DNS server is usually provided by your ISP. This also has a local cache of recently looked up domains. If a result is found locally, then sent it back to your PC & your request ends. Else, move on to the Root DNS Server for an answer.
3. The Root servers act as the DNS backbone of the internet -- their job is to redirect you to the correct TLD server (by identifying the parts correctly).
4. TLD Server holds records for where to find the authoritative server (aka NameServer) to answer the DNS req. Eg: NS for tryhackme.com is kip.ns.cloudflare.com. Many NS exists as backups too.
5. Autho-DNS is responsible for storing all DNS records for a particular domain name & is also where any updates to your DNS records would be made. Depending upon the record type, the DNS record is then sent back to the Recursive DNS server, where a local copy will be cached for future requests & then relayed back to the original client that made the request. DNS records come with a TTL value (time to live - unit : seconds) representing how long a response be saved locally before having to look it up again -- caching saves time.

HTTP in Detail

Set of rules used for communicating with webservers for transmitting webpage data (HTML, images,videos, etc).

Requests & Responses
URL is needed first. Has many parts like scheme (http/s), port, fragment, query string, etc.
A req is made via "GET" > `GET / HTTP/1.1` -- Req Method & Protocol version.
Req has many fields. Firstly the Method (GET), then the 'Host', 'User-Agent', 'Referrer', etc.
Resp would include Protocol version & the Status Code ('200 OK'), also includes Server, Date, Content-Type, Content-Length and then the information requested, which is usually the body.

HTTP Methods
GET : getting info from the web-server
POST : submitting data to it -- creating new records
PUT : submitting data to it -- updating some info
DELETE : delete any info/records

HTTP Status Codes
(visit https://http.cat for more info)
Range -
100-199 : Info -> not common anymore
200-299 - Success
300-399 - Redirection
400-499 - Client Errors
500-599 - Server side errors
Common Codes -
200 - OK
201 - Created
301 - Permanently moved
302 - Found
400 - Bad Req
401 - Not Authorised (username pw needed)
403 - Forbidden (not allowed, no matter logged in or not)
404 - Not Found
405 - Method not allowed
500 - Internal service error
503 - Service unavailable

Headers
Common Req Headers ->

Host, User-Agent, Content-Length, Accept-Encoding, Cookie.
Common Resp Headers ->
Set-Cookie, Cache-Control, Content-Type, Content-Encoding

Cookies
Saved when you receive a 'Set-cookie' header. HTTP is stateless, cookies can be used to remind the web-server who you are (+/ your settings). Those settings are sent in the Set-Cookie header with proper key-value. Then from now on, all req will be come back with this cookie data. In case of passwords, they are usually a hashed token.

How Websites Work

2 parts -> Front-end (the way our browser renders the site)
& Back-end (server that processes the req & returns a resp)

HTML
Websites are made using HTML, CSS & JS.
HTML is tags and attribute based.
JavaScript
Implemented inside the `<script>` tag usually, this is the dynamic code on a webpage.
Example : `<script src="/location/of/javascript_file.js"></script>`
OR
`<button onclick='document.getElementById("demo").innerHTML = "Button Clicked";'>Click me!</button>`
//Sensitive Data Exposure happens when a website forgets to remove clear-text sensitive info in the source code -- which is accessible to anyone.

HTML Injection
Happens when websites fail to filter any malicious text that a user might input into a website. This way, the attacker can inject malicious HTML code onto the site -- a client side thing.
Input Sanitisation is a necessary practice in this case.

Web-Stuff Put Together

The flow : Req website in your browser -> Find web server IP addr with DNS -> Connect to that web-server -> view website.
Other Components

1. Load Balancers : A traffic handler when the main web-server is over-loaded. It sits in front of all the web-servers. It ensures high traffic websites can handle the load & providing a failover if server becomes unresponsive. It uses algos like round-robin (sent to each in turn) or weighted (find the least busy one). They can also perform health-checks on all servers.

2. CDNs : Content Delivery Network, useful for cutting down traffic to a busy website. It allows you to host static files from your website (JS, CSS, Img, MP4,..) & host them across many servers all over the world. When a user requests a file, the CDN finds the nearest server & sends the request there instead of anywhere else.
3. Databases : Webservers can communicate with DB to store & recall data from them. DB can range from simple plain-text files to complex clusters. Eg: MySQL, MSSQL, MongoDB, etc.
4. WAF : Web-Application Firewall -- sits b/w the web request & the websever. Its primary purpose is to protect the webserver from any hacking/DDoS attempt. It analyses the web-requests for common attack techniques (bot or real user) + checks for excessive requests by using 'rate-limiting' (certain amount of IPs per second). Sus req are dropped.
   How Web-servers work
   A web-server listens for incoming connections & then utilises HTTP to deliver web-contents to its clients. Common examples include Apache, Nginx, IIS & NodeJS. A web-server always returns the content placed in its root directory. Nginx/Apache use `/var/www/html` in Linux and IIS uses `C:\inetpub\wwwroot` for Windows.

Virtual Hosts : WS can host multiple websites with difft domain names -- so they use VH for this. VH are just text based config files. Example : one.com being mapped to `/var/www/website_one` & two.com being mapped to `/var/www/website_two`.

Static vs Dynamic Content : Static never changes i.e pictures, javascript, CSS, but can also include HTML that never changes. Files are directly served from WS with no changes made.
Dynamic is the opposite. Latest entries, updates on a frequency, search query on a blog, etc. Changes done in the Backend but ofc reflected onto the Frontend.

Scripting & Backend Language : PHP, Python, Ruby, NodeJS, Perl, etc. Superb languages.

An example of a full rundown of the steps involved :
Request website in your browser -> Check local DNS cache for IP Addr -> Check your recursive DNS Server for Addr -> Query root server to find authoritative DNS server -> Authoritative DNS server advises the IP Addr for the website -> Req passes through a WAF -> then req passes through a load balancer -> Connect to the WS on port 80 or 443 -> WS receives a GET req -> Web App talks to the DB -> Finally, your browser renders the HTML into a viewable website.

Cybersecurity 101 Modules

Windows Command Line

Many commands.
Remember, most commands will comply with `/?` flag to see their help page.

- `set` command will show all the Path variables of how Windows will execute commands.
- `ver` will show the Windows version.
- `systeminfo` will show the OS details.
- `help` & `cls` are nothing new.
- `more` command is applicable here as well.
- `driverquery` for instance is a big command showing all driver info but you can pipe it with more -> `C:\> driverquery | more`
- `ipconfig` for basic network queries, and `ipconfig /all` for a detailed view
- `ping example.com` & `tracert example.com` both are known
- `nslookup example.com` (you can add the IP after the domain to force the nameserver).
- `netstat` is responsible for showing current active connections & listening ports. This command has many arg flags like `-a` (all established connections & listening ports), `-b` (programs associated w/ each listening port & estb connection), `-o` (reveals the PID with the connection), `-n` (numerical form to addresses & port numbers). Combine them all at once and it becomes -> `C:\> netstat -abon`.
- `cd` & `dir` are known. `dir` has `/a` (all-display hidden files as well) & `/s` (displays current dir & all sub-dirs) flags.
- `tree` command also carries over.
- `mkdir`, `rmdir`, `copy`, `move`, `del` or `erase`, `type` (for file-type). Wildcards will also work here. Eg: `copy *.md C:\Markdown` will copy all markdown files to that directory.
- `tasklist` is basically the `ps aux` command from Linux. To see all available filters of this command use `tasklist /?`. One such is `FI` stands for 'filter' followed by any condition if you want. Eg : `C:\> tasklist /FI "imagename eq sshd.exe"`.
- `taskkill` is used to kill a process using its PID. Same as `kill` in linux.
  Eg : `C:\> taskkill /PID target_pid`.
- `chkdsk` a very important command. So is `sfc /scannow`. Similarly, `shutdown` with `/s` to actually shutdown, `/a` to abort a shutdown, `/t` for a timer, `/fw` for entering firmware & `/r` to restart are useful.

Windows Powershell

Object oriented. Built on the .NET framework.

Basics

Powershell uses `cmdlets` called command-lets. Powerful than traditional windows commands. They follow a verb-noun naming convention. Verb is the action, noun is the object. Example : `Get-Content` or `Set-Location`. Writing '`Get-Command`' will list all the use-able commands. You will notice that the output will have 4 object as the columns (CommandType, Name, Version, Source). It is always possible to filter the output of any cmdlet based on the objects displayed by it. Here we can filter for 'functions' by using
`PS:> Get-Command -CommandType "Function"`.

Use `Get-Help` followed by the target command to see the man page of the target command.
Example : `PS:> Get-Help Get-Date`. You can also append `-examples` to get a better explanation using examples.
Ofc to make life easier, all commands have an alias (shortcut names). You can see all of them using `Get-Alias`. Example : cat -> Get-Content; cd -> Set-Location, pwd -> Get-Location. Many cmdlets have more than 1 aliases. cat, type, gc -- all point to Get-Content.

Powershell can be expanded with more cmdlets that can be downloaded from outside. Format is `Cmdlet -Property "Pattern*"`. Example : `PS:> Find-Module -Name "Powershell*"`. This can be followed up with Install-Module, like `PS:> Install-Module -Name "PowerShellGet"`.
Using `PS:> Get-Alias -Name "echo"` will only reveal the command that echo is an alias of, which is 'Write-Output'.

Navigating the File System

Just like `dir` we have `ls` and `Get-ChildItem`, to list everything.
`Set-Location` with the Path flag can be used to 'cd' into the target directory.
Example : `PS:> Set-Location -Path "./Documents"`

To create an item in PS, we use `New-Item` followed by -Path + path_value + -ItemType + file/directory. Example :
`PS:> New-Item -Path ".\SomeFolder\OneMoreFolder" -ItemType "Directory"`
-- this will create 'OneMoreFolder' inside SomeFolder.
Now to make a txt file inside we can use :
`PS:> New-Item -Path ".\SomeFolder\OneMoreFolder\my-file.txt" -ItemType "File"`

Removing is a tad-bit different. CMD will have `del` and `rmdir`, separate for file and folders but here, `Remove-Item` is same for both. These 2 are valid :

```
PS:> Remove-Item -Path ".\SomeFolder\OneMoreFolder\my-file.txt"
PS:> Remove-Item -Path ".\SomeFolder\OneMoreFolder\"
```

Copy & Move is done by `Copy-Item` & `Move-Item`.

```
PS:> Copy-Item -Path .\Closet\Shelf\coat.txt -Destination .\Closet\Shelf\coat2.txt
PS:> Move-Item -Path .\Closet\Shelf\coat2.txt -Destination .\Closet\
```

[Make sure you're not in the Closet or Shelf directory while performing these. If you are then remove the name of the parent directory.]

`cat` is replaced by `Get-Content` here. Altho cat will also work. Example :

```
PS:> Get-Content -Path ".\coat.txt"
```

Piping, Filtering & Sorting Data

Piping works in the same manner as in Linux. Output of one command is sent as input for another command. Example :

```
PS:> Get-ChildItem | Sort-Object Length
```

OR

```
PS:> Get-ChildItem | Where-Object -Property "Extension" -eq ".txt"
```

(-eq, -gt, -ge, -lt, -le, -like)

```
PS:> Get-ChildItem | Where-Object -Property "Name" -like "ship*"
```

Can also select kinda like SQL

```
PS:> Get-ChildItem | Select-Object Name,Length
```

To display the largest file, you can chain the cmdlets

```
PS:> Get-ChildItem | Sort-Object -Descending | Select-Object -First 1
```

Lastly, Select-String can search for pattern within files (same as grep/findstr).

```
PS:> Select-String -Path ".\coat.txt" -Pattern "thrift"
```

System & Network Information

`Get-ComputerInfo` (akin to `systeminfo` from CMD).

`Get-LocalUser` to list all the user accounts on the system.

`Get-NetIPConfiguration` & `Get-NetIPAddress` -- are both meant for ipconfig stuff.

Real-Time System Analysis

`Get-Process` is kinda like ps-aux to see all the processes like scvhost, winlogon, etc.

`Get-Service` to see all the services running -- like Xbox, Wifi, Defender etc.

`Get-NetTCPConnection` will debug current TCP connections.

`Get-FileHash` is for getting the hash of a file (usually accompanied with -Path and file's path)

Scripting

An essential command is `Invoke-Command`. Meant for efficient remote management, and when combined with scripting, it can help in automation of tasks across multiple machines. Can also be used to execute payloads or commands. Example ->

```
PS:> Invoke-Command -FilePath C:\scripts\test.ps1 -ComputerName Server01
```

OR
`PS:> Invoke-Command -ComputerName Server01 -Credential Domain01/User01 -ScriptBlock { Get-Culture }`
OR - Running Get-Service on a PC without uname/pw
`PS:> Invoke-Command -ComputerName RoyalFortune -ScriptBlock { Get-Service }`

Linux Shells

Same old boring stuff.
pwd, cd, ls, cat, grep, more, less, history -- all done in Linux Basics for Hackers book.
Talking about shells ->
Bash (Bourne again shell) : sorta old, wide in scripting, basic customization, less user-friendly, no syntax highlighting.
Fish (friendly interactive shell) : Limited scripting, advanced tab completion, good customization, most user friendly, syntax highlighting.
Zsh (Z shell) : Top level of scripting (bash + extra features), tab completion can be extended via plugins, Advanced customization (can make super user friendly), syntax highlighting (but via plugins).
Next is scripting, executing the script, for loops, input in scripts, if-else, comments, etc.

Networking Concepts

OSI Model
Layers - 1 to 7 - Phy, Data Link, Network, Transport, Session, Presentation & Application.
Important Protocols in each layer :
1 - Physical - Electrical/Optical/Wireless
2 - Data Link - Ethernet (802.3) & WiFi (802.11)
3 - Network - IP, ICMP, IPSec
4 - Transport - UDP, TCP
5 - Session - NFS, RPC
6 - Presentation - Unicode, MIME, JPEG, PNG, MPEG
7 - Application - HTTP, FTP, DNS, POP3, SMTP, IMAP
Common Questions ->
Responsible for E2E comms b/w running apps - Layer 4
Responsible for routing packets to proper network - Layer 3
Responsible for encoding application data - Layer 6
Responsible for Data b/w hosts on the same network - Layer 2

TCP/IP Model
4 - Application Layer -- stores 3 Layers (5,6,7) of OSI -- HTTP/s, FTP, POP3, SMTP, IMAP, Telnet, SSH.

3 - Transport Layer -- same (L4) -- TCP/UDP
2 - Internet Layer -- Network Layer of OSI (L3) -- IP, ICMP, IPSec
1 - Link Layer -- Data Link Layer (L2) -- 802.11, 802.3

IP Addresses & Subnets
ifconfig, subnets, 4 octets from 0-255, each of 8 bits making 32 bits in total for 1 IP.
`192.168.1.0` is the network address & `192.168.1.255` is broadcast address.
Private IPs can only be in range with `10.0.0.0 - 10.255.255.255`, `172.16.0.0 - 172.31.255.255` & `192.168.0.0 - 192.168.255.255`
i.e `10/8` , `172.16/12`, `192.168/16` [The unit after / determines the leftmost bits that do not change. Total 8 * 4 = 32. /8 meaning only first 8 bits i.e first octect (10.) is constant. 16 means first 2 octects, 12 means first + half of second.]

UDP & TCP
(Same basics) In UDP, port 0 is reserved. Rest is same -- Port 1 to 65535 (2^16 - 1).
UDP is stateless/careless/rebellious. TCP is not. TCP needs a 3-way handshake.
Both are Layer 4 protocols.

Encapsulation
Process of every layer adding a header (& at times a trailer) to the received unit of data & sending this 'encapsulated' unit to the layer below.
Happens in 4 steps ->
a) Application Data : User input. An email or an Instant message for example.
b) Transport Protocol segment/datagram : Transport layer (TCP/UDP) adds the proper header info & creates TCP segment (or UDP datagram). This segment is sent to the layer below it, the network layer.
c) Network Packet : Internet layer, adds an IP Header to the received TCP segment or UDP datagram. Then this IP packet is sent to the layer below it, the DL layer.
d) Data Link Frame : Ethernet/Wifi receives the packet & adds the proper header & trailer, creating a frame.
Application Data ->
TCP/UDP Header + Application Data
IP Header + TCP/UDP Header + Application Data
Eth/Wifi Header + IP Header + TCP/UDP Header + Application Data + Eth/Wifi Header

Networking Essentials

DHCP
Whenever we want to access a network - 3 things are configured : IP Addr + Subnet Mask, Router (gateway) & DNS Server. This happens whenever anyone connects to a new network.
DHCP is the one automating this process cuz you don't carry your domain controller in your pocket. It also saves us from address conflicts.

DHCP relies on the UDP port 67-68. Server listens on 67 and client sends from 68.
4 Steps - D-O-R-A
Discover - Offer - Request - Acknowledge
Flow : Client broadcasts a DHCP-DISCOVER message, seeking the local DHCP server if one exists. Server responds with a DHCP-OFFER w/ an IP Addr available for client to accept. Client responds with a DHCP-REQUEST message to indicate that it has accepted the offered IP and finally Server responds back with a DHCP-ACK message to confirm that this IP addr is now assigned to the client.
Example :
`$:> tshark -r DHCP-G5000.pcap -n`
Output :
`1 0.000000 0.0.0.0 → 255.255.255.255 DHCP 342 DHCP Discover - Transaction ID 0xfb92d53f`
`2 0.013904 192.168.66.1 → 192.168.66.133 DHCP 376 DHCP Offer - Transaction ID 0xfb92d53f`
`3 4.115318 0.0.0.0 → 255.255.255.255 DHCP 342 DHCP Request - Transaction ID 0xfb92d53f`
`4 4.228117 192.168.66.1 → 192.168.66.133 DHCP 376 DHCP ACK - Transaction ID 0xfb92d53f`
Client starts with a no IP, only MAC. After sending DISCOVER, the DHCP server at `192.168.66.1` replies with an offer. Even till step 3, the client has no IP as it needs to use the IP offered by DHCP. So it sends packets from the IP Addr `0.0.0.0` to the broadcast `255.255.255.255`. In the end, DHCP does provide the IP and client accepts it.
DHCP uses Client's MAC address to handle all this. So in essence, DHCP has leased the IP Addr to the client to access n/w resources, given the gateway to route our packets outside the local network & given a DNS server to resolve the domain names.

ARP
Bridging layer 3 addressing to layer 2 addressing.
As 2 hosts communicate over a network, an IP packet is encapsulated within a data link frame as it travels over layer 2. Common DL layers are Eth (802.3) & Wifi (802.11).
For the proper DL header, IP is not enough, we also need the Target's MAC.
Devices on same eth n/w do not need to know each other's MAC all the time -- only while communicating, without which there's nothing to communicate. Everything otherwise revolves around IP Addrs.
Example : You connect to a n/w. If this n/w has a DHCP server, your device will be auto configured to use a specific gateway (router) + a DNS server. Now, your device knows the IP of the DNS server to resolve any domain name + it knows the IP Addr of the router for when it needs to send packets over the internet. In all this scenario, no MAC Addrs are revealed.
Every IP Packet will have a SRC MAC ADDR, a DST MAC ADDR, and TYPE (IPv4)

ARP makes it possible to find the MAC of another device on the ethernet.
What happens is -> A device sends an ARP req with it's IP Addr enclosed, asking for some other device's IP. The ARP req is sent from the sender's MAC to the broadcast addr i.e `ff.ff.ff.ff.ff.ff`. Then the ARP reply arrives, and the other device replies back with it's MAC saying it is attached to the requested IP. Now, these 2 can exchange DL layer frames. (You can use `tcpdump` or `tshark` to see the flow -- or inspect it in Wireshark.)

ICMP
Internet Control Message Protocol.
`ping` & `traceroute` / `tracert` are important troubleshooting commands.
Ping :
Sends an ICMP ECHO req (type 8). THe receiving end replies with a type 0 ECHO message.
Altho, a FW might stop ping requests.
Traceroute :
IP has a field called TTL which indicates the max number of routers a packet can travel through before it is dropped. Router decrements the packet's TTL by 1 before it sends it across. When TTL == 0, router drops the packet & sends an ICMP Time Exceeded Msg (Type 11) -- time measured in number of routers, not seconds.
Hence, traceroute can determine the path a packet took to travel from SRC to DST.
You can always look up those addresses (public/private) that might show up in the output.

Routing
Everything hops through multiple routers on the internet to reach their destination. Routers are essential in connecting multiple networks. Routing Protocols & Algos decide the proper flow. Protocols ->

1. OSPF - Open shortest path first : share info on the n/w topology & calculate the most efficient path for transmission. Here, routers exchange updates about the status of their connected links & n/ws so that each router has a complete map of the n/w & determine best routes.
2. EIGRP - Enhanced Interior Gateway Protocol : Cisco proprietary routing protocol. Allows routers to share info about the n/ws they can reach + the cost associated with those routes.
3. BGP - Border Gateway Protocol : Allows difft n/ws (like those of ISPs) to exchange routing info & establish paths for data travel b/w these n/ws + it helps ensure data can be routed efficiently even when traversing multiple n/ws
4. RIP - Routing info protocol : Often used in small networks. Routers here share info about the n/ws they can reach + number of hops reqd. Result - each router builds a routing table based on this info, choosing the routes with fewest hops.

NAT
Network Address Translation.
IPv4 can support a max of 4 billion devices. Many private IPs can work under 1 public IP. That's the idea behind NAT. This way you only need to provide Internet access to just 1 publicly exposed IP. (works in 2^ i.e 2 public IPs for 30 private ones = 32).
Unlike simple routing, routers that support NAT must find a way to track ongoing connections. Also, NAT-supporting routers maintain a table translating n/w addrs b/w internal & external n/ws. (Internal n/w could use a Private IP range and external can do with Public IP).
In any normal home setup with PCs, Laptops, Cameras, etc; the Router will maintain a table that maps the internal IP & PORT of all these devices w/ it's external IP Addr & Port.
Example : 3 devices with IP Addr & Port as -> `192.168.0.129:15401` , `x.x.x.137:27912` , `x.x.x.112:38192` -> will all be seen externally with `212.3.4.5` but separate ports `:19273` , `:32759` , `:41251` by the webserver.
[Theoretically, a router with infinite processing power can handle 65000 simultaneous TCP connections]

Networking Core Protocols

Ports summary ->
TELNET : TCP : 23
DNS : TCP/UDP : 53
HTTP : TCP : 80
HTTPS : TCP : 443
FTP : TCP : 21
SMTP : TCP : 25
POP3 : TCP : 110
IMAP : TCP : 143

DNS
Remembering Addresses. We don't have to. Bcz of DNS. As read previously, We have 4 records in DNS :
A Record - Address - mapping a hostname to one/more IPv4 Addr. Eg: `something.com` resolves to `172.17.2.172` .
AAAA Record - Same but for IPv6.
CNAME - Canonical Name - maps domain to another domain. Eg: `example.com` mapped to `www.example.com` or even `example.org` .
MX Record - Mail Exchange - specifies the Mail Seever responsible for handling emails for a domain.
`$ > nslookup` is useful for this. This command will send DNS queries back and forth, including one A and one AAAA record query.

WHOIS

Domain name is resolved to an IP Addr. But for this, someone needs to have the authority to set the A,AAAA & MX Records, among other DNS records for the domain. This power lies with the one who registers the domain. All the info can be had via `whois` because everyone needs to give info while registering and it goes in the public db.

HTTP(S)

S is for secure. Common methods are `GET`, `POST`, `PUT` & `DELETE`.
HTTP on Port 80 and HTTPS on Port 443. (8080 is a backup for Port 80).
Wireshark can be used to see the basic structure of the web page. Burp suite and other tools are also handy.
Telnet was mentioned but it's better to also try other commands like `wget` and `curl`.
A) `$ > telnet 10.48.146.30`
`GET /flag.html HTTP/1.1`
`Host: anything`
B) `$ > wget http://10.48.146.30/flag.html` --> followed by `$ > cat flag.html`
C) `$ > curl http://10.48.146.30/flag.html -o flag.html` --> followed by `$ > cat flag.html`

FTP

Efficient and quick. Has its own commands like `USER`, `PASS`, `RETR`, `STOR` that run behind the scene, akin to the SYN/ACK stuff of TCP. Observing via Wireshark will reveal the same.
FTP can handle a few unix commands like `ls`. You can change the file type for text files, lets say for ASCII by using `type ascii`.
`get` command can be used to fetch a file. Example : `$ > get flag.txt`.

SMTP

How a mail client talks to a mail server. By default listens on TCP Port 25.
SMTP has its own set of commands that run ->
`HELO` or `EHLO` - initiates SMTP session
`MAIL FROM` - specifies the sender's email address
`RCPT TO` - specifies the receiver's email addr.
`DATA` - indicates that client will begin sending the content of the email message
`.` - is sent on a line by itself to indicate the end of email message.

POP3

Receiving email. Aptly named 'Post Office Protocol'.
In a primitive manner, you can connect on TCP port 110 via telnet
`$ > telnet _IP-HERE_ 110`
If SMTP is handing your letters to the post box, POP3 is basically checking your own mailbox outside your house.

POP3 Commands :
`USER <uname>`
`PASS <passwd>`
`STAT` - req the number of messages & total size
`LIST` - list all messages & their sizes
`RETR <message_number>` - retrieves the specified message
`DELE <message_number>` - marks the message for deletion
`QUIT`

IMAP
Synchronizing mail across devices/sessions. Internet Message Access Protocol.
Listens on TCP Port 143 by default.
Commands ->
`LOGIN <uname> <passwd>` - authenticates
`SELECT <mailbox>` - selects the mailbox folder to work with.
`FETCH <mail_number> <data_item_name>` - Eg: fetch 3 body[]
`MOVE <sequence_set> <mailbox>` - moves the specified messages to another mailbox
`COPY <sequence_set> <data_item_name>` - copies the specified messages to another mailbox.
`LOGOUT`

Networking Secure Protocols

Secure protocols & securing existing ones.

TLS
Initially SSL was developed -- secure socket layer -- a crypto protocol to secure the communication b/w a client & a server. Later on, TLS 1.0 was built on SSL 3.0. These crypto protocol operate at OSI model's Transport layer. Secure means C&I - Confidentiality & Integrity. TLS ensures no one can read or modify the exchanged data.
Existing protocols did upgrade via TLS :
HTTP -> HTTPS; DNS -> DoT (DNS over TLS); MQTT -> MQTTS & SIP -> SIPS; SMTP -> SMTPS, & POP3S, IMAPS, etc.
In summary,
A server/client needs to identify itself to get a signed TLS cert. Usually the server admin creates a CSR -- Cert Signing Req -- and submits it to the CA -- Cert Authority -- then CA verifies the CSR & issues a digital cert. Once the signed cert is rec'd, it can be used to identify the client/server to others - who can confirm the validity of the sign.
For a host to confirm the validity of a signed cert, the certs of signing authorities need to be installed on the host. [Usually a fee is involved in getting a cert signed however we do have free alt's like Let's Encrypt].
PS: Self-signed certs should never be used to confirm the authenticity of a server.

HTTPS

Consider this - a client requests a webpage over HTTP in a browser. After resolving the domain name to an IP Addr -> Before the actual GET Request, the 3-way TCP handshake takes place (SYN, SYN-ACK, ACK), post which the actual web-page is fetched via GET / HTTP/1.1 and after that, connection is closed via TCP (FIN-ACK, FIN-ACK, ACK).

HTTPS is basically HTTP over TLS. Now, requesting a page over HTTPs will require 3 steps (after resolving the domain nam) : Establish a TCP 3-way handshake with the server -> Establish a TLS session -> Communicate using the HTTP protocol.
Now, in any sample case, there will be 3 packets for the TCP handshake and 4-5 packets for the TLS negotiation. These 2 sets combined make for the proper 'TLS Negotiation & Establishment' packets. After these, we get the Application Data packets (because there's no way to know if it's indeed HTTP or some other protocol).
Now following the packets and combining them (via Wireshark) will reveal a scrambled file.

Adding TLS over HTTP makes the packets encrypted. Cannot see without a private key -- not that you will ever have that key.

New Ports summary ->
HTTP : TCP : 80
HTTPS : TCP : 443
SMTP : TCP : 25
SMTPS : TCP over TLS : 465 & 587
FTP : TCP : 21
FTPS : TCP w/ TLS Cert : 990
POP3 : TCP : 110
POP3S : TCP over TLS : 995
IMAP : TCP : 143
IMAPS : TCP over TLS : 993

SSH

Port 22.
SSH offers not only password-based auth but also public-key & 2FA.
E2E provided + notifies you of new server keys to protect against MITM.
Confidentiality of the Data + Cryptography protects the integrity of the traffic.
Tunneling is possible hear to route other protocols through SSH (kinda like VPN).
X11 Forwarding can help in a GUI over-the-network.

```
$ > ssh username@hostname
```

-X flag can help with X11 GUI forwarding

```
$ > ssh root@192.168.1.38 -X
```

SFTP & FTPS
FTP & FTP-Secure which uses TLS. Uses Port 990 unlike FTP who uses 21.
Caveat? Unlike HTTPS, SMTPS etc who only want TLS for security, FTPS requires a proper TLS cert to run securely.

SFTP is actually SSH-FTP. Different than FTP/S. Uses same port 22 as SSH.
Connect via `$ > sftp user@host` followed by commands like `get filename` & `put filename` to download/upload files.
Setting up SFTP is relatively easy as you can enable it within the OpenSSH server.

VPN
TCP/IP had good enough rules for routing and sending the packets but no solution for securing all data from disclosure/alteration. Hence why we have VPNs.
Simple client-server direct tunnel connection. Good for companies with many branches in different places.
Once established, all traffic is routed over the VPN i.e through the tunnel. Anyone on the outside will not see the client's IP but rather the VPN server's IP -- hence the geo restriction bypass -- and local ISPs only see the encrypted traffic.
(Not all would be this good. Be careful of DNS leak by some providers).

Wireshark Basics

2 main file types - `.pcap` and `.pcapng`
Very versatile and robust tool. Only reads the packets -- doesn't/cannot modify them.
Ofc you have tools in the toolbar for stuff like sniffing & processing.
A display filter bar for the actual packet filtering + can set many capture filters.
After opening a file - We can observe the packet list + packet bytes in 2 ways - Hex & decoded ASCII. We also have a packet + comment count from the file.
(You can ofc set coloring rules depending upon the packet type).
You can even merge files together -- by visiting the merge menu.
To see the file properties - either go to Statistic -> Capture File Properties OR by clicking a notepad icon bottom left (has properties like File Comment, Stats, interfaces, etc).
3 sections - Packet Dissection, Navigation & Filtering.

Packet Dissection

- Also called 'Protocol Dissection', which investigates packet details by decoding available protocols & fields. Wireshark already has many -- you can also write your own.
- When you hover over the packets first line ("frame 33, 54 bytes on wire")-- which is usually the description -- the whole packet bytes code (hex) is highlighted. Then hovering over subsequent lines we can see which source is highlighted ("Ethernet

II SRC", "IPv4 SRC", "TCP SRC", "Protocol Errors", etc). These categories mostly align to the OSI model layers -- working top to down (Refer to THM Room images).

Packet Navigation

- All packets are numbered ofc. You can go int the 'Go' sub-menu and hit 'Go to Packet 'and enter the number.
- We also have a 'Find Packet' menu that can find packet based on some filters -- via Display filter, Hex, Strings & Regex. Search fields like Packet list, packet details & packer bytes BUT these are separate panes. Wireshark will know nothing on what goes on under one filter.
- You can even Mark the packets (Ctrl+M or Edit>Mark/unmark : default color is always black).
- You can add comments (Ctrl Alt C or Edit>Packet comment) on the packet.
- Packets can be exported to (File>Export) with file type options.
- 2 Display formats -> Time display Default and UTC Time display. (View>Time Display Format)
- Expert Info : A proper mode. (Analyze > Expert Info)
  Example Cae : Go to a packet by typing its number, find a packet containing '.txt'; Export a packet's JPEG byte code and use MD5Sum to see the hash; etc.

Packet Filtering
2 types - capture & display filters.
Cap Filters are used for 'capturing' only the packets valid for the used filter.
Disp Filters are used for 'viewing' the packets valid for the used filter.

- Apply as filter : The most basic way to filter traffic. You can right click any captured packet and use that as a filter for the whole pcap file. Selecting any packet to be applied as filter will result in us having control of only 1 entity of the packet (ip.src or tcp.stream).
- Prepare as Filter : Just prepare the said filter, don't apply it yet.
- Conversation Filter : To expand on basic filtering. It helps you view only related packets & hide the rest easily. It expands on the filter (lets say, to include ip.src but also port, etc).
- You can then, optionally, after filtering, Colorize those conversations. View > Colorize Conversation (and a reset button is there as well). This will keep everything highlighted.
- Apply as Column : You can put any packet detail/info and make that as a column (Eg: Packet Details > Ethernet > Type : IPv4 -> Apply as column and it appears as expected).

- Follow Stream : Click on a packet - Rt Click > Analyze > Follow Stream (TCP/UDP). Useful when you want some form of a trail. A separate window with color coded data shall appear, wherein red is client originated data and blue is server originated data. Wireshark places the filter automatically in this case.

TCP-Dump Basics

`tcpdump` uses libpcap & OpenSSL libraries. Basically Wireshark but CLI/lacking GUI. You can run it as is, but ofc, running it while it is hooked to an interface is better. We are civilized. We won't run it naked. We have flags to attach.

- `-i INTERFACE_NAME` or `-i any` -- to listen on all available interfaces. Eg: `-i eth0`
- `-w FILE_NAME` can be used to save the dump data to a .pcap file -- although you won't see the packets while using this flag.
- To read such files, you can use `-r FILE_NAME`.
- `-c COUNT` can be used to specify the number of packets to capture. Without it, it goes on and on till you interrupt it.
- TCPDump will default resolve IP Addrs & print use-able domains when possible. To avoid DNS lookups, use the `-n` flag & use `-nn` flag to stop both DNS and port number lookups. Basically, we see the numeric data instead of the fancy words i.e direct IP values.
- Lastly, `-v` flag will produce more verbose outputs. However, tcpdump also offers a `-vv` and `-vvv` for higher and higher verbosity.
- Examples : `$ > tcpdump -i eth0 -c 50 -v` ; `$ > tcpdump -i wlo2 -w data.pcap` OR even `$ > tcpdump -i any -nn`.

Filtering

- Filtering by Host : You only wanna see packets from a certain domain/IP, we can use the `host` keyword. Eg : `$ > sudo tcpdump host example.com -w sample.pcap`. This will only fetch the data relevant to his domain by only capturing 'conversations' done with this domain.
  HOWEVER you also have keywords like 'src' and 'dst' to prefix the host keyword `src host IP_ADDR` & `src host HOSTNAME` -- to limit packets to those originating from this source. Similarly you can limit packets in the file to the ones sent to a particular destination by using `dst host IP_ADDR` or `dst host HOSTNAME`.
- Filtering by Port : Use the `port` keyword followed by the port number to limit the captured packets to those on this particular port. Example : DNS uses TCP and UDP ports 53 by default -- so we'll read all DNS queries read by our NIC.
  Eg : `$ > sudo tcpdump -i en0 port 53 -n` -- will query both IPv4 and IPv6 (A and AAAA records from DNS).

- Filtering by Protocol : Protocols can be filtered -- keywords like `ip`, `ip6`, `udp`, `tcp` & `icmp`. Example : Using icmp and seeing 'echo' means someone is running the `ping` or `traceroute` command.
  `$ > sudo tcpdump -i eth0 icmp -n`.
- Logical Operators : A few that can be used to club conditions. We can use `and`, `or`, `not`, etc.
  Examples : `$ > tcpdump host 1.1.1.1 and tcp` ; `$ > tcpdump udp or icmp` ; `$ > tcpdump not tcp`;
  `$ > tcpdump -i any tcp port 22` -- listen on all interfaces + capture tcp packets to/from port 22 i.e SSH
  `$ > tcpdump -i wlo1 udp port 123` -- listen on Wifi & filter UDP traffic to port 123 (i.e NTP - network time protocol).
  `$ > tcpdump -i eth0 host example.com and tcp port 443 -w https.pcap` -- will listen on eth0 and filter traffic exchanged with example.com that uses tcp port 443.
  //Remember to use the `$ > wc` - word count command with '|' operator to get a count & '-n' to avoid unnecessary delays in attempting to resolve IP Addrs. Also, reading doesn't require sudo. `Eg: $ > tcpdump -r traffic.pcap src host 192.168.124.1 -n | wc`
  Few Cases -> To filter all packets on ICMP : `$ > tcpdump -i ens5 -r traffic.pacp icmp -n` and optionally use `... | wc` to get the count.
  -> To see the IP of the host that's inquiring about the MAC of 192.168.124.137 :
  `$ > tcpdump -i ens5 -r traffic.pcap dst 192.168.124.137 and arp -n` -- because IP to MAC mapping is done via ARP.
  -> DNS queries : `$ > tcpdump -r traffic.pcap port 53 | head`

Advanced Filtering
Many more filters.
`greater LENGTH` will filter packets that have a length gr8r than or = & `less LENGTH` will filter that have a length less than or =.
[Try the 'pcap-filter' command. See its manual.]
[Binary Operations : ! , & , | for NOT,AND,OR]
Header Bytes : Protocols like ARP, ICMP, IP etc. We need a proper method to filter for these protocols. Since pcap-filter is technically embedded in tcpdump, we can use its syntax -> `proto[expr:size]`.
`proto` can be `arp`, `ether`, `icmp`, `ip`, `ip6`, `tcp` & `udp`
`expr` is the byte offset, 0 refers to the first byte.
`size` is the number of bytes that interest us. Can be 1,2 or 4. Optional and default is 1.
Examples ->
a) `ether[0] & 1 != 0` -- takes the first byte in the ethernet header & the decimal

number 1 (`0000 0001 in binary`) and applies the '&' operation. Purpose is to show packets sent to a multicast addr [Multicast Eth addr identifies a group of devices intended to receive the same data].
b) `ip[0] & 0xf != 5` -- takes the first byte in the IP header & compares it with hex number F (`0000 1111 in binary`). Will return true if the result is != to decimal 5. Purpose is to catch all IP packets with Options.
c) `$ > tcpdump -r file.pcap greater 15000 -n -t` -- will filter packets with length > 15000 bytes.
Our focus is TCP FLAGS. Which is done via `tcp[tcpflags]` . This will gather all the flags in all packets and then you can compare against other flags individually.
Available TCP flags : `tcp-syn` , `tcp-ack` , `tcp-fin` , `tcp-rst` (reset) & `tcp-push`
So we can write, to capture ->
`$ > tcpdump "tcp[tcpflags] == tcp-syn"` -- packets with **only** `SYN` flag set, all other flags unset.
`$ > tcpdump "tcp[tcpflags] & tcp-syn != 0"` -- packets with **at least** `SYN` flag set.
`$ > tcpdump "tcp[tcpflags] & (tcp-syn|tcp-ack) != 0"` -- packets with **at least** `SYN` or `ACK` flags set.

Displaying Packets
Some useful more flags to play with.
`-q` : quick output/print brief packet info. (IPs and timestamps)
`-e` : print link-level header (IPs, MACs, type, length etc.)
`-A` : show packet data in ASCII
`-xx` : show packet data in hex format
`-X` : show packet headers & data in hex and ASCII
Eg: `$ > tcpdump -r file.pcap -q`

Nmap Basics

Scans are done on IP or hostnames. Either one single IP, or a range like `192.168.0.1-10` or a whole subnet like `192.168.0.1/24` -- which is same as writing `192.168.0.0-255` .
Without sudo is an unprivileged scan -- aka 'connect' scan.
`-sn` -> ping scan
`-sL` -> this followed by the IP subnet range will reveal the IPs that will be scanned as a list.
`-sT` -> TCP connect scan -- full 3-way handshake.
`-sS` -> Syn scan (stealth) -- handshake is not completed.
`-sU` -> scan for all UDP services -- DNS,DHCP, NTP, SNMP etc.
`-F` -> fast mode -- scans the 100 most common ports (instead of default 1000)
`-p[range]` -> specify the ports -- eg: -p10-1024, -p-25 (1-25), -p- (all).

`-O` -> OS detection
`-sV` -> service & its version
`-A` -> above 2 + more stuff
`-Pn` -> scan hosts that appear to be down
`-T VALUE` OR `-T keyword` -> -T0 for paranoid, -T1 for sneaky, 2 for polite, 3 normal, 4 aggressive & 5 is insane. `-T4` same as `-T aggressive`.
`--host-timeout` -> max amt of time to wait for target host
`--min-parallelism <numprobes>` -> min no. of parallel probes
`--max-parallelism <numprobes>` -> max no.
`--min-rate <number>` & `--max-rate<number>` -> Min and Max rate (packets/sec)
`-v` -> verbose output ; `-vv` , `-vvvv` same as `-v2` and `-v4` for higher verbosity.
`-d` -> debugging level output. Levels range from 1 to 9 -> `-d9`
`-oN <filename>` -> Normal output
`-oX <filename>` -> XML output
`-oG <filename>` -> grep-able output
`-oA <basename>` -> Output in all major formats.
Eg: `$ > nmap -sS 192.168.139.1 -oA gateway`

Cryptography Basics

In the most simplest manner, the 'data' is encrypted using a 'key'. Sender converts the 'plaintext' data to 'ciphertext' data by doing so. Then, this key is also used on the receiving end to decrypt the ciphertext data and obtaining the plaintext data.
Usually, cipher is publicly known but the key must remain a secret (unless it is the public key in asymm encryption).
Primer
Plaintext : TRYHACKME
Key : 3 -- assuming a right shift of 3 while encrypting(T -> ~~U~~,~~V~~,W)
Cipher : Caser Cipher
Ciphertext : WUBKDFNPH
(So a left shift of 3 while decrypting)
Types of Encryption
Symmetric :

- aka 'Private Key Cryptography' -- single key --keeping the key a secret, is a must -- which is a challenge -- need a secure accessible channel.
- Case : Emailing a secure doc is easy, but emailing the key is risky. Send it once and it's not a secret anymore. You need a secure accessible channel again.
- DES (Data Encryption Standard) -- 1977; 56bit key ; 1999 broken, 3DES -- 168 bits (3 times DES) but effectively 112 bits -- ad hoc solution ; deprecated in 2019 & AES -- 2001 ; 128,192 or 256 bits.
  Asymmetric :

- aka 'Public Key Cryptography' -- uses a pair of keys, one to encrypt other to decrypt.
- Eg : RSA, D-H, ECC (elliptic curve), etc.
- A public key and a private key. One can be kept known, other is kept hidden.
- Usually slower -- cuz of larger keys in the ciphers. RSA uses one out of 3 : 2048/3072/4096-bit keys. D-H also uses a min of 2048-bit but also uses the other 2 for better security. ECC can do the same with shorter keys like : 256-bit which is comparable to 3072-bit RSA key.
- Basically a math problem -- easy to compute in 1 direction but impossible in reverse.
  Math Operations
  2 main Ops - XOR and Modulo
  Modulo :
  `A % B` -- returns the remainder of A divided by B. Eg: `25%5 = 0`.
  Irreversible. Always returns a non-negative result less than the divisor. For `int a` and a +ve `int n`, the result of `a%n` will always be in the range of `0` to `n-1`.
  XOR :
  'exclusive OR' -- returns `0` if the bits are same (either 1 or 0), and returns `1` when bits are different.
  Eg : 1001 ⊕ 1010 = 0011 (compute one by one from left to right digit by digit)
  XOR is useful for crypto and error detection. Basically, A ⊕ A = 0 and A ⊕ 0 = A --> it means, XOR leaves the value as is when you -xor- with 0 and returns 0 when you -xor- that value with itself.
  Also, A ⊕ B = B ⊕ A , and (A ⊕ B) ⊕ C = A ⊕ (B ⊕ C) --- commulative + associative.

Example : P is plaintext, C is ciphertext and K is key. If we know C & K, we can recover P.
We start with C ⊕ K = (P ⊕ K) ⊕ K. But we know that (P ⊕ K) ⊕ K = P ⊕ (K ⊕ K) because XOR is associative. Furthermore, we know that K ⊕ K = 0; consequently, (P ⊕ K) ⊕ K = P ⊕ (K ⊕ K) = P ⊕ 0 = P. In other words, XOR served as a simple symmetric encryption algorithm. In practice, it is more complicated as we need a secret key as long as the plaintext.

A	B	A ⊕ B
0	0	0
0	1	1
1	0	1

A	B	A ⊕ B
1	1	0